MDS_Assignment1_313652018_Xuan-Wei_Wang

March 25, 2026

1 Five Stage Analytics

1.1 Wafer Map Detective Problem

This problem typically occurs in semiconductor manufacturing lines (such as etching or lithography processes). In advanced manufacturing, the defect data on a Wafer Map can vary significantly. Some wafers have specific fatal defects, such as "Scratches" or "Edge-Rings." These patterns make it difficult for traditional machine learning classifiers to work correctly, often leading to "False Negatives" (missing a real defect). When defective wafers slip through to the next stage of production, it can cause massive financial losses for the factory.

To solve this, we aim to introduce Generative AI based on mechanisms like Denoising Score Matching. By generating more data for these rare defect types, we can balance the dataset. This will greatly improve the accuracy of our prediction models and help maintain a higher production yield.

1.2 Five Stage Analytics

It all begins with **Descriptive Analytics**. By collecting sensor data and Wafer Maps from Automated Optical Inspection (AOI), the system can monitor and visualize the current yield status in real-time. When descriptive indicators show specific abnormal distributions—such as rare but fatal scratches or edge-ring defects—we can trace the alerts back upstream to find the "problem maker" and the root causes of these patterns. This naturally moves the process toward **Diagnostic Analytics**, such as identifying a gas flow imbalance in a specific etching machine.

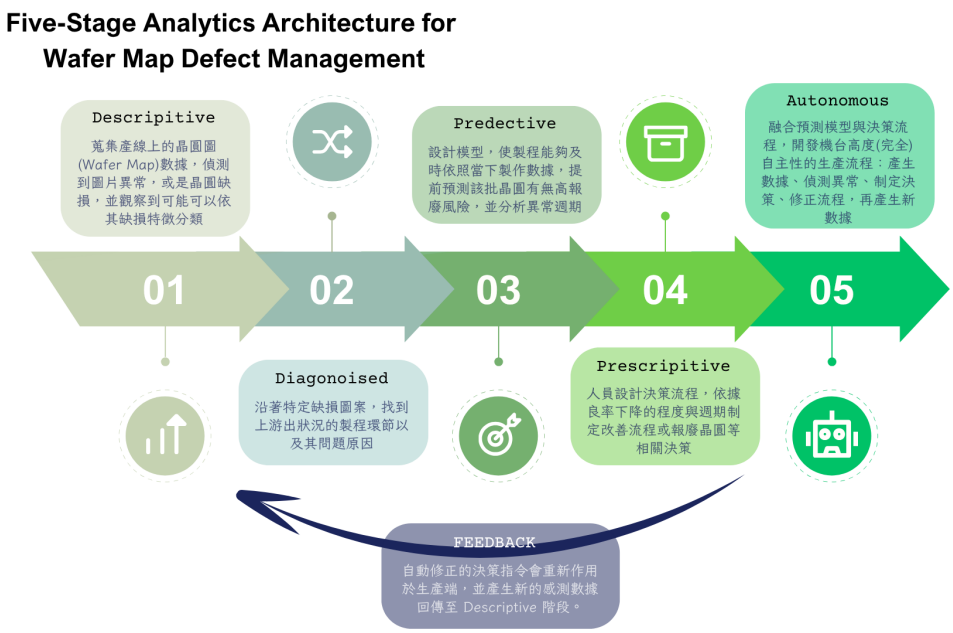
Once the root cause is found, we can design models to predict if a batch of wafers has a high probability of being scrapped based on real-time streaming data. This pushes the process into **Predictive Analytics**. For example, we can introduce Diffusion Models to generate high-quality defect images to expand our dataset. This enhanced data is then used to train machine learning classifiers, enabling them to "predict" high scrap risks and gain insights into anomaly cycles even when a machine has only a slight deviation.

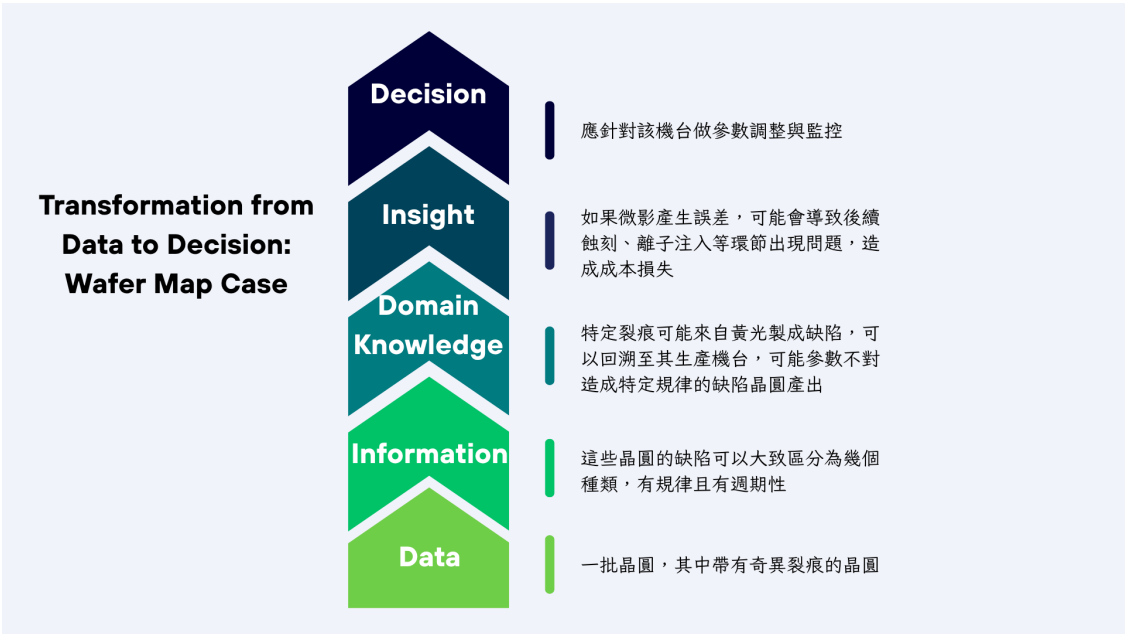
If the system detects a high scrap probability, we must consider the associated costs, such as production downtime losses and recycling costs, or go back a step to adjust parameters. Therefore, the data flow must seamlessly connect to **Prescriptive Analytics**. This means factory personnel design a decision system that doesn't just show yield warnings, but also balances maintenance costs against scrap losses to provide the best "prescription"—such as suggesting an immediate machine stop or fine-tuning specific process parameters.

Finally, when the suggestions from the prescriptive system are proven highly reliable, the ultimate goal of this decision chain is **Autonomous Analytics**. In this stage, the system removes human decision-makers from the loop and establishes a closed-loop Advanced Process Control (APC)

system. The machines can then perform real-time, autonomous parameter compensation and corrections based on predictions and prescriptions. These automated production results generate new data, which feeds back into the initial "Descriptive Analytics" stage for monitoring, achieving a fully unmanned, self-correcting closed-loop system.

1.3 Five-Stage Analytics Architecture for Wafer Map Defect Management





1.4 Academic Paper Report

1.4.1 (1) Wafer Map Defect Patterns Classification Based on Convolutional Neural Network

Ref: Wafer Map Defect Patterns Classification Based on Convolutional Neural Network (Nakazawa & Kulkarni, 2018)

This paper focuses on using CNNs to classify wafer map defect patterns and establishing a searchable knowledge base for wafer defects to help production engineers. Specifically, it includes:

1. **Defect Pattern Classification**: How to automatically and accurately classify abnormal defect features on wafers (such as Ring, Scratch, and Cluster defects).
2. **Establishing an Effective Knowledge Base**: Once wafer maps are linked to their root causes in a knowledge base, engineers can quickly locate the source of the problem when similar defects occur in the future.
3. **Data Imbalance Problem**: Addressing the difficulty of obtaining enough training data for rare defect patterns in actual manufacturing processes.

These innovations were designed to solve technical problems at the time, such as:

- **Rapid Response to Process Issues**: Traditional manual identification or old feature extraction methods lacked efficiency and accuracy. The industry needed a more automated way to quickly monitor abnormal signals and respond to process issues.
- **Image Retrieval Efficiency**: How to quickly retrieve wafers with similar defect patterns from a massive historical wafer map database to facilitate follow-up analysis.
- **Solving Data Scarcity During Technology Development**: During the development phase, there is often a lack of sufficient real defect data to train deep learning models. One of the motivations of this study is to prove that using synthetic data to train models can also effectively identify real wafer defects, thereby improving the detection capability for rare events.

This research mainly covers the following three stages:

- Descriptive Analytics: Visualization and Classification. Wafer map defect patterns are visualized, and CNNs are used to automatically classify 22 defect categories (e.g., C1 random defects, C2 ring defects, etc.).
- Diagnostic Analytics: Root Cause Identification. Through the Image Retrieval function, similar patterns are searched within historical databases. By comparing current cases with historical examples of known Root Causes, engineers can diagnose the reasons behind current process anomalies.
- Predictive Analytics: Failure Characteristic Prediction. The CNN model performs Inference on unseen wafers to predict their defect category and the probability of occurrence.

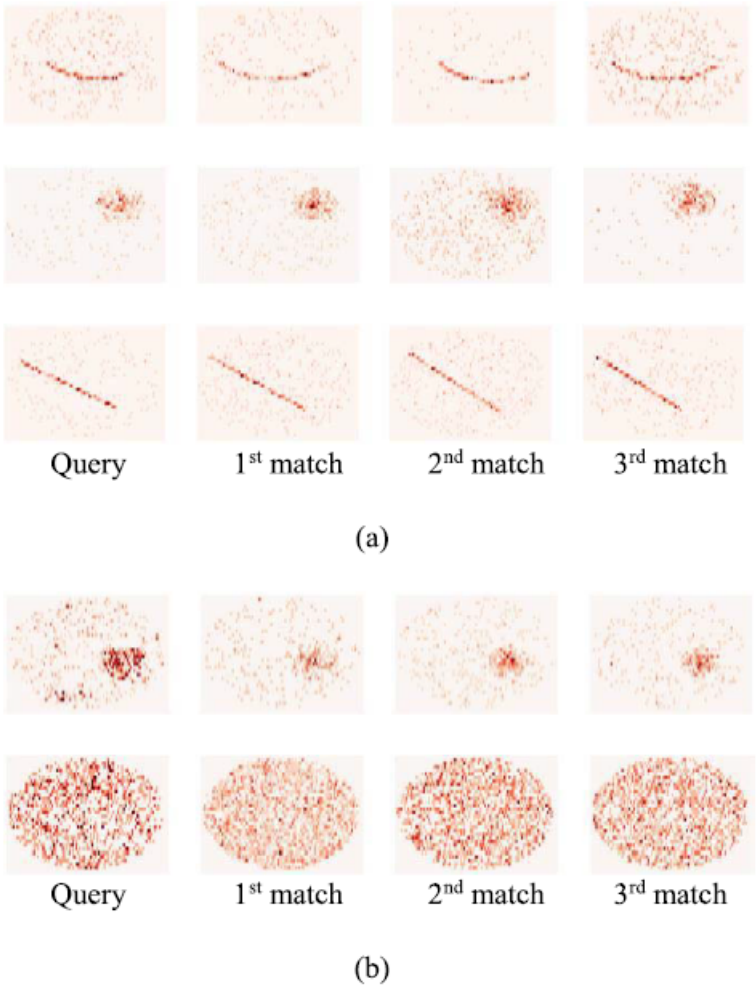


Fig. 8. Query wafer map (1st column) and the corresponding top 3 retrieved wafer map images for the selected defect patterns. (a) Query wafer map is from simulation. (b) Query wafer map is from the real wafer.

The methodology of this provides a key triggering mechanism for "Information → Domain Knowledge".The paper mentions that once root causes and solutions are linked to specific wafer map patterns, the Image Retrieval system can be used to trigger actions directed at the problematic process. This optimizes the decision-making process for engineers to determine when and how to repair equipment.

In addition, this research achieves an end-to-end model approach, which does not require manual development of specific feature extractors or domain expert knowledge. This automates the inspection process that originally required manual identification, enabling the system to actively identify rare defect patterns (Rare event detection), especially during the technology research and development phase.

> *Without having enough number of dataset, CNN cannot be trained well and it is difficult to have enough data size in some cases if defect patterns happen rarely. Our model enables rare event detection capability without having real data and it is particularly beneficial during technology development phase.* ((Nakazawa & Kulkarni, 2018))

1.5 Critical Thinking

I believe a particularly unique point is that since defective wafers account for a very small percentage of total production, or are even rare, the sample size is often insufficient to support this kind of research. To generate samples, they used the Poisson point process:

$$P(k, \Lambda) = \frac{\Lambda^k}{k!} e^{-\Lambda}$$

In this formula, the parameter Λ determines the average proportion of defective wafers in a segment, and k is the number of events.

However, with the development of Generative AI, this traditional method has become quite unique in comparison.In a real factory environment, however, situations are more complex, and the Poisson point process method has its limits. For example, if there are specific noise or defect patterns in the actual process (such as edge effects or specific distributions caused by chemical reactions), the model's performance will decrease. It might be better to simulate the physical characteristics of a specific machine in the process that leads to certain defect clusters to generate samples. Although the parameters or the process would be more complex, the performance would likely be better than using purely mathematical methods. Alternatively, using Generative AI, whether through Diffusion models or Flow matching, and incorporating physical characteristics to generate samples would be much more convenient.

2 Manufactoring System Analysis

2.1 Manufactoring Information

Throughput (TH) is the number of good products produced per unit of time, so we can obtain:

Station	Parallel Machines (m)	Process Time (t)	Capacity TH_i (units/hr)
1	3	5	$3/5 = 0.6$
2	3	8	$3/8 = 0.375$
3	6	12	$6/12 = 0.5$
4	2	4	$2/4 = 0.5$
5	5	12	$5/12 \approx 0.417$

Based on the table above, Station **2** has the smallest capacity, making it the bottleneck station. * Bottleneck Throughput (r_b): $r_b = 0.375$ (units/hour) * Minimum Cycle Time (T_0): This is the total processing time, also known as Raw Process Time (RPT). * $T_0 = 5 + 8 + 12 + 4 + 12 = \mathbf{41}$ (hours) * Critical Work-In-Process (W_0): The minimum WIP level required for the system to achieve maximum throughput with zero waiting time. * $W_0 = r_b \times T_0 = 0.375 \times 41 = \mathbf{15.375}$ (units) * Line Stability Condition: The Release Rate (R) must be less than the bottleneck throughput. * Stability Condition: $R < r_b$ (i.e., $R < 0.375$)

2.2 Best Case

- Cycle Time(CT)

$$CT(w) = \begin{cases} 41, & \text{if } w \leq 15.375 \\ w/0.375, & \text{otherwise.} \end{cases}$$

- Throughput(TH)

$$TH(w) = \begin{cases} w/41, & \text{if } w \leq 15.375 \\ 0.375, & \text{otherwise.} \end{cases}$$

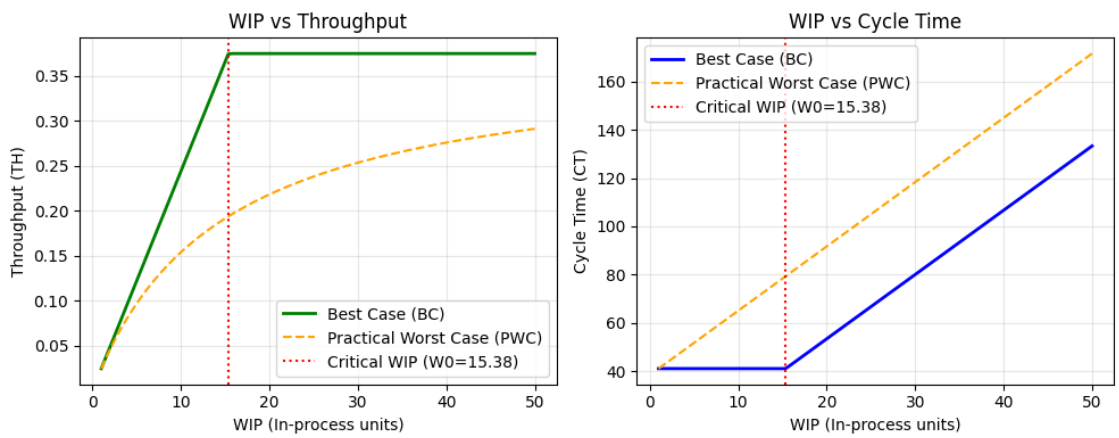
2.3 Manufacturing System Simulation

We will generate two types of plots to demonstrate the relationships among the three key variables: WIP vs. Throughput (TH) and WIP vs. Cycle Time (CT).

[1]:
```
import numpy as np
import matplotlib.pyplot as plt

m = [3, 3, 6, 2, 5]
t = [5, 8, 12, 4, 12]
th_stations = [m[i]/t[i] for i in range(len(m))]

rb = min(th_stations)
T0 = sum(t)
W0 = rb * T0

w_values = np.linspace(1, 50, 100)

th_best = np.array([min(w/T0, rb) for w in w_values])
ct_best = np.array([max(T0, w/rb) for w in w_values])

th_pwc = (w_values / (W0 + w_values - 1)) * rb
ct_pwc = T0 + (w_values - 1) / rb
```

[3]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

# WIP vs Throughput (TH)
ax1.plot(w_values, th_best, label='Best Case (BC)', color='green', lw=2)
ax1.plot(w_values, th_pwc, label='Practical Worst Case (PWC)', color='orange',␣
↪ls='--')
ax1.axvline(x=W0, color='red', ls=':', label=f'Critical WIP (W0={W0:.2f})')
ax1.set_xlabel('WIP (In-process units)')
ax1.set_ylabel('Throughput (TH)')
ax1.set_title('WIP vs Throughput')
ax1.legend()
ax1.grid(True, alpha=0.3)

# WIP vs Cycle Time (CT)
ax2.plot(w_values, ct_best, label='Best Case (BC)', color='blue', lw=2)
ax2.plot(w_values, ct_pwc, label='Practical Worst Case (PWC)', color='orange',␣
↪ls='--')
ax2.axvline(x=W0, color='red', ls=':', label=f'Critical WIP (W0={W0:.2f})')
ax2.set_xlabel('WIP (In-process units)')
```

```
ax2.set_ylabel('Cycle Time (CT)')
ax2.set_title('WIP vs Cycle Time')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Bottle neck rate rb = {rb} units/time")
print(f"If Release Rate (R) > {rb}, system will enter unstable state (WIP␣
↪accumulates infinitely).")
```




```
Bottle neck rate rb = 0.375 units/time
If Release Rate (R) > 0.375, system will enter unstable state (WIP accumulates
infinitely).
```

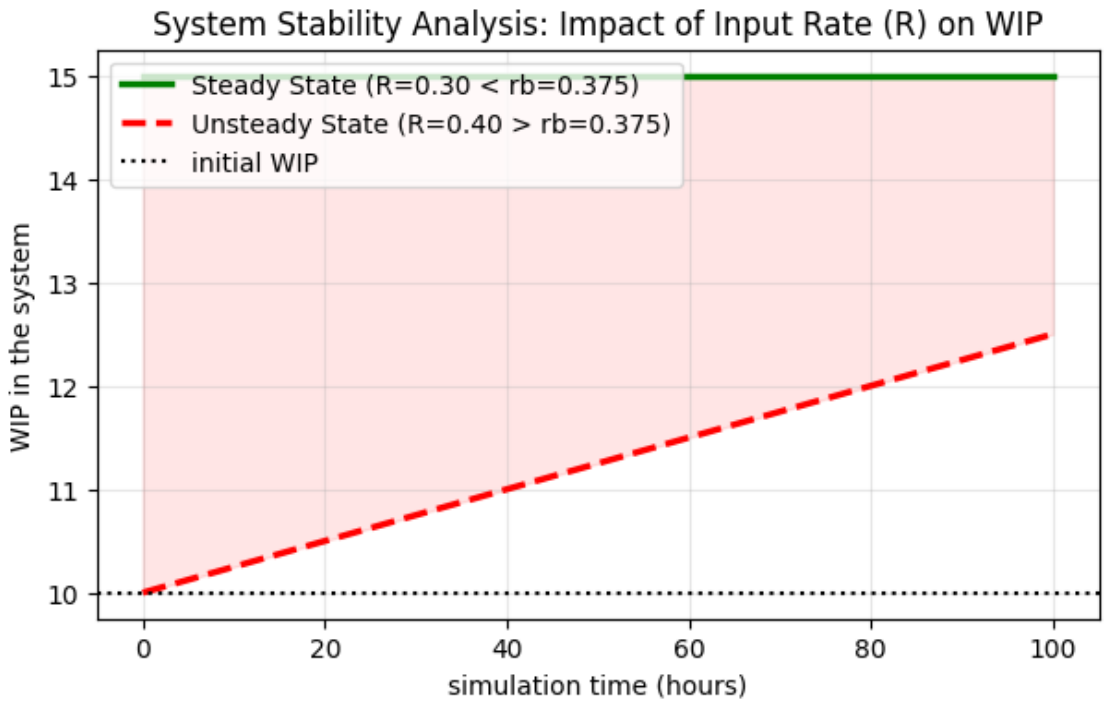
2.3.1 Relationship between WIP, CT, and TH

- TH (Throughput): When $WIP < W_0$, TH increases as WIP increases. However, once WIP exceeds W_0 (15.375), the throughput becomes limited by the bottleneck station and stays at 0.375. After this point, adding more WIP will not increase capacity.
- CT (Cycle Time): When $WIP < W_0$, CT remains near T_0 (41 hours). But once WIP exceeds W_0, CT begins to increase linearly in direct proportion to WIP.

[4]:
```
R_steady = 0.30
R_unsteady = 0.40

time_steps = np.linspace(0, 100, 101)

WIP_initial = 10

wip_steady_profile = np.full_like(time_steps, WIP_initial + 5)
```

```
wip_unsteady_profile = WIP_initial + (R_unsteady - rb) * time_steps
```

[5]:
```
plt.figure(figsize=(7, 4))
plt.plot(time_steps, wip_steady_profile, label=f'Steady State (R={R_steady:.2f}␣
↪< rb={rb:.3f})',
         color='green', lw=2.5)
plt.plot(time_steps, wip_unsteady_profile, label=f'Unsteady State␣
↪(R={R_unsteady:.2f} > rb={rb:.3f})',
         color='red', lw=2.5, ls='--')

plt.axhline(y=WIP_initial, color='black', ls=':', label='initial WIP')
plt.fill_between(time_steps, wip_steady_profile, wip_unsteady_profile,␣
↪color='red', alpha=0.1)
#plt.text(80, wip_unsteady_profile[-1]-5, 'WIP infinity accumulation',␣
↪color='red', fontweight='bold')

plt.xlabel('simulation time (hours)')
plt.ylabel('WIP in the system')
plt.title('System Stability Analysis: Impact of Input Rate (R) on WIP')
plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)

plt.show()

print(f"bottleneck output rate rb = {rb}")
print(f"unsteady input rate R = {R_unsteady}")
print(f"WIP accumulation rate = R - rb = {R_unsteady - rb:.4f} (units/hour)")
```




```
bottleneck output rate rb = 0.375
unsteady input rate R = 0.4
WIP accumulation rate = R - rb = 0.0250 (units/hour)
```

2.3.2 Release Rate and Throughput (TH)

When the Release Rate (R) exceeds the bottleneck throughput (r_b) (as shown by the red line), the system cannot maintain a steady state, and WIP (Work-In-Process) accumulates linearly over time. This leads to a dramatic increase in queuing time, which in turn causes the Cycle Time (CT) to deteriorate significantly (represented by the pink area).

1. Green Line (Steady State): Represents the WIP level the system should maintain under healthy, stable conditions.
2. Red Line (Unsteady State): Represents the actual WIP remaining in the system when the release rate is too high.
3. Pink Area: The distance between these two lines represents the "Excess Queue" generated because $R > r_b$.

In other words, we can draw the following conclusions: * Steady State: Occurs when $R < r_b$. In this state, the system's throughput (TH) equals the release rate (R) ($TH = R$). Whatever goes in comes out; the system is in balance. * Non-steady State: Occurs when $R > r_b$. In this state, the system's throughput (TH) is capped at the bottleneck limit ($TH = r_b = 0.375$). Because input is faster than output, the system cannot reach an equilibrium.

3 Reference

1. Takeshi Nakazawa and Deepak V. Kulkarni, *Wafer Map Defect Pattern Classification and Image Retrieval Using Convolutional Neural Network*, IEEE TRANSACTIONS ON SEMICONDUCTOR MANUFACTURING, VOL. 31, NO. 2, MAY 2018